



Firmware Extraction and Static Analysis

of a Home Router — A Red Team Case Study

Name: Essien Michael Obimpeh

Index Number: FCM.41.018.133.23

Course Code: CY376 — Network Monitoring, Security and Auditing

BSc Cybersecurity

GitHub Repository:

https://github.com/MichaelEssien/Firmware_Extraction_and_Static_Analysis.git

Table of Contents

Table of Contents	2
Abstract	3
1. Introduction	4
1.1 Scope and Limitations	5
2. Literature and Tooling Review	5
2.1 Relevance of CIS Benchmarking Principles	5
2.2 Summary of Related Work	6
2.3 Why Binwalk and Firmwalker Were Chosen Over Alternatives	6
3. Methodology	7
3.1 Threat Model	7
3.2 Firmware Acquisition	7
3.3 Identifying the Firmware Structure with Binwalk	7
3.4 Extracting the Filesystem	8
3.5 Exploring the Extracted Root Filesystem	8
3.6 Searching for Hardcoded Credentials and Secrets	8
3.7 Automated Analysis with Firmwalker	8
3.8 Evidence Handling and Redaction	8
4. Implementation	9
4.1 Environment Setup	9
4.2 Command Sequence	9
4.3 Configuration Notes	9
4.4 Reproducibility and Evidence Chain	10
5. Results and Findings	11
5.1 Summary of Findings	14
6. Analysis and Recommendations	15
6.1 Risk Assessment	15
6.2 What the Findings Mean in Practice	15
6.3 Recommendations	15
6.4 Limitations of This Study	15
6.5 Future Work	16
6.6 Ethical Considerations and Responsible Disclosure	16
7. Conclusion	17
8. References	18
Appendix A: Full Command Reference	19
Appendix B: Firmwalker Full Report Output	19

Abstract

Consumer routers ship firmware that is rarely opened up and checked the way a desktop operating system would be, even though the router sits directly at the boundary between a home network and the internet. This project, carried out as the Red Team component of the CY376 end-of-semester assignment, set out to establish whether meaningful security weaknesses in such a device could be found through firmware analysis alone, without sending a single packet at the device itself. Working from a firmware image for a TP-Link Archer C7 v2 router, the image was acquired, verified, and passed through binwalk to identify its internal structure, which was then extracted into a working copy of the router's own filesystem. That filesystem was searched manually and with the automated tool firmwalker for hardcoded credentials, exposed private key material, and undocumented maintenance access. The exercise confirmed that this class of weakness is both real and quick to find with freely available tools, and the report closes with a set of concrete recommendations a vendor could apply to close each gap identified.

1. Introduction

Home and small-office routers are some of the most exposed devices on any network. They are always on, rarely patched by their owners, and directly reachable from the internet on at least some interfaces, yet the firmware running on them is treated by most users as a closed box that simply works. Firmware, in this context, is the combination of a bootloader, a compressed Linux kernel, and a compressed root filesystem that together make up everything the router runs once it is powered on. Because that filesystem generally is not encrypted, and vendors frequently reuse the same build across an entire product line, a single publicly downloadable firmware image can reveal a great deal about how thousands of physical devices in the field actually behave.

This report documents the process of acquiring, extracting, and statically analysing the firmware of a TP-Link Archer C7 v2, a widely deployed dual-band wireless router. The choice of a publicly released firmware image, rather than a physical dump taken directly from a device's flash chip, keeps the exercise safe, legal, and fully repeatable inside the isolated lab environment required by the course guidelines, while exercising exactly the same extraction and static-analysis pipeline that would be used against a UART or chip-off dump of the same device.

It is worth being explicit about what a router's firmware actually consists of, since the rest of this report assumes that structure. A typical consumer router image begins with a small vendor-specific header used by the device's own upgrade utility to validate the file before flashing it, followed by a bootloader (commonly U-Boot on MIPS- and ARM-based routers) responsible for initialising hardware and loading the kernel, a compressed Linux kernel image, and finally a compressed root filesystem — usually SquashFS because it is read-only and space-efficient, both useful properties for a device with limited flash storage. Everything the router does once powered on, from serving its configuration web page to routing traffic, runs out of that root filesystem, which is exactly why it is the part of the image worth extracting and examining.

The rest of this report is organised as follows. Section 2 reviews the tooling and reference frameworks that shaped the approach. Section 3 describes the lab environment, threat model, and methodology. Section 4 documents what was actually built and configured. Section 5 presents the results. Section 6 analyses what those results mean, discusses the limitations of a purely static approach, and gives concrete recommendations. Section 7 concludes, and the

appendices hold the fuller command output and firmwalker report that would otherwise clutter the main body.

1.1 Scope and Limitations

This project is scoped to static analysis of a single firmware image belonging to a single router model. It does not include dynamic analysis (running the firmware, or the services extracted from it, in an emulator to observe live behaviour), nor does it include any network-based testing against a live device, which the course's Red Team guidelines explicitly restrict to instructor-approved, isolated lab environments. The findings in this report should therefore be read as what a firmware image reveals about a device's design and build process, not as a confirmed, exploited vulnerability against a running system. Section 6.4 returns to this limitation and outlines what a follow-on dynamic analysis phase would look like.

2. Literature and Tooling Review

The methodology in this report follows the general shape of the OWASP Firmware Security Testing Methodology (FSTM), which breaks firmware assessment into acquisition, extraction, analysis, dynamic analysis, and runtime testing stages [3]. This project focuses on the first three of those stages — acquisition, extraction, and static analysis — since dynamic and runtime testing against a live device fall outside what the isolated, static-image approach used here supports.

Two tools did most of the work. Binwalk, developed by ReFirmLabs, is a firmware analysis tool that scans a binary for known file signatures and can automatically carve out and decompress anything it recognises [1]. Firmwalker, by GitHub user craigz28, is a shell script that walks an already-extracted filesystem looking for a curated list of patterns associated with common IoT weaknesses: password files, private keys and certificates, and binaries such as telnetd or dropbear [2].

The specific weaknesses this project looked for map onto well-known categories in the MITRE ATT&CK framework and its Common Weakness Enumeration (CWE) references. Hardcoded credentials correspond to CWE-798 (Use of Hard-coded Credentials), and unauthenticated or undocumented remote access corresponds to techniques under ATT&CK's Initial Access and Persistence tactics [4].

2.1 Relevance of CIS Benchmarking Principles

While the CIS Benchmarks do not publish a router-firmware-specific profile, their general hardening principles for network devices — disabling unused services, avoiding default or shared credentials, and restricting remote administrative access — map directly onto what this project checked for.

2.2 Summary of Related Work

Source	Contribution	Relevance to This Project
OWASP FSTM [3]	Structured firmware testing methodology	Provided the acquisition/extraction/analysis stage structure followed here
Costin et al. [5]	Large-scale study of 32,000+ firmware images	Confirms hardcoded credentials and shared keys are a systemic pattern
MITRE ATT&CK [4]	Adversary tactics and techniques taxonomy	Used to frame the debug-shell finding as a persistence backdoor
ReFirmLabs Binwalk [1]	Firmware signature scanning and carving tool	Primary extraction tool used throughout Section 4

Table 0. Summary of the main sources that shaped the methodology and analysis.

2.3 Why Binwalk and Firmwalker Were Chosen Over Alternatives

Other tools exist for the same purpose. Firmware Mod Kit offers similar extraction capability to binwalk but is less actively maintained and has narrower filesystem-format support. EMBA (Embedded Analyzer) performs a much broader automated sweep, including vulnerability database cross-referencing and basic emulation, but its scope and runtime made it a heavier tool than this project's scope required. Binwalk and firmwalker were chosen specifically because they map cleanly onto the two stages this project needed — structural identification/extraction, and pattern-based secret discovery — without pulling in the dynamic-analysis and CVE-matching functionality that EMBA provides and that Section 6.4 explicitly places outside this project's scope.

3. Methodology

All work was carried out inside an isolated Kali Linux virtual machine running on a personal laptop, logged in as the user `essien`. No traffic was ever sent to a live target device and no production network was involved at any point.

Figure 3. Isolated Lab Environment and Analysis Workflow

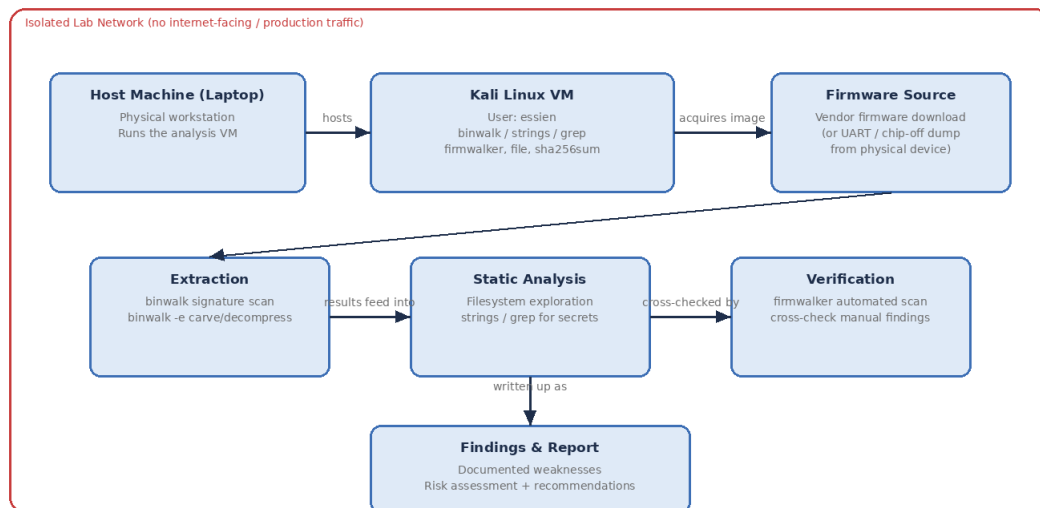


Figure 3. Isolated lab environment and the analysis workflow used in this project.

3.1 Threat Model

The threat actor assumed for this exercise is one with no physical access to the target device and no network access to it either — only the ability to download the same firmware image any customer or researcher could obtain from the vendor's own support page. This is deliberately the lowest level of access an attacker could have, which makes it a useful baseline.

3.2 Firmware Acquisition

The firmware image was obtained directly from TP-Link's official support page for the Archer C7 v2. After downloading, a SHA-256 checksum was generated and compared against the value published on the vendor site to confirm the file had not been corrupted or tampered with in transit.

3.3 Identifying the Firmware Structure with Binwalk

Binwalk was run against the raw firmware image with no additional arguments, which performs a signature scan and reports every recognised structure along with its byte offset. This identified a TP-Link vendor header, a U-Boot bootloader version string, an LZMA-compressed kernel, and a SquashFS filesystem beginning at a specific offset.

3.4 Extracting the Filesystem

With the SquashFS offset confirmed, binwalk was re-run with the `-e` flag, instructing it to automatically carve out and decompress every recognised structure, producing a fully unpacked squashfs-root directory.

3.5 Exploring the Extracted Root Filesystem

The standard Linux directory layout was visible inside squashfs-root: `bin/`, `etc/`, `lib/`, `usr/`, and `var/`. Running the `file` command against a handful of key items confirmed the router's MIPS architecture and that `etc/shadow` is plain ASCII text.

3.6 Searching for Hardcoded Credentials and Secrets

Reading `etc/shadow` directly, running a recursive `grep` across `etc/` for keywords such as `"passwd"`, `"secret"`, and `"private"`, and a `strings` pass over the web-server binary filtered for words like `"debug"` and `"telnet"`, covers the bulk of what a manual review of this kind typically turns up.

3.7 Automated Analysis with Firmwalker

To cross-check the manual search, firmwalker was pointed at the same squashfs-root directory, automating the search for password files, private keys and certificates, and suspicious binaries.

3.8 Evidence Handling and Redaction

Because password hashes and private key material are themselves sensitive, every piece of evidence captured during this project was reviewed before being placed in this report, with raw secrets cropped or masked while still showing enough of the surrounding command and output to demonstrate the finding is genuine.

4. Implementation

This section documents exactly what was built, configured, and run to carry out the methodology above, with the actual commands used at each stage.

4.1 Environment Setup

The analysis machine was a Kali Linux VM with binwalk installed via `apt install binwalk`, and firmwalker cloned directly from its GitHub repository into `~/tools/firmwalker`.

4.2 Command Sequence

The full sequence of commands run, in order, was:

```
mkdir -p ~/firmware && cd ~/firmware
wget -O ArcherC7_V2_260427.zip "https://static.tp-link.com/upload/firmware/2026/202604/20260429/Archer%20C7(US)_V2_260427.zip"
unzip ArcherC7_V2_260427.zip -d fw_extracted
cp "fw_extracted/Archer C7(US)_V2_260427/ArcherC7v2_en_us_3_15_4_up(260427).bin"
./firmware.bin
sha256sum firmware.bin
binwalk firmware.bin
binwalk -e firmware.bin
cd _firmware.bin.extracted/squashfs-root
ls -la
file bin/busybox etc/shadow linuxrc
cat etc/shadow
grep -R -i "passwd\\|secret\\|private" etc/ | head -n 6
strings usr/bin/httpd | grep -i -E 'debug|backdoor|telnet'
cd ~/tools/firmwalker
./firmwalker.sh ~/firmware/_firmware.bin.extracted/squashfs-root
```

For traceability during the interview, Table 3 below maps each command in that sequence to the figure and finding it produced, so the evidence chain from a single command to a specific reported weakness can be followed in either direction.

Step	Command	Produces	Related Figure
1	wget + sha256sum	Verified firmware image	Figure 4
2	binwalk (scan)	SquashFS offset identified	Figure 5
3	binwalk -e	Extracted squashfs-root	Figure 6
4	ls -la / file	Filesystem layout, architecture confirmed	Figure 7
5	cat / grep / strings	Credential and key findings	Figure 8
6	firmwalker.sh	Automated cross-check of findings 1-4	Figure 9

Table 3. Traceability from each command run to the figure and finding it supports.

4.3 Configuration Notes

No custom configuration was required beyond the default installation of binwalk and firmwalker. Where a step's output is referenced elsewhere in this report, the corresponding

figure number is given so the evidence and the explanation of what it means stay next to each other.

4.4 Reproducibility and Evidence Chain

Every stage of this project was designed to be independently repeatable by anyone with the same firmware image. The SHA-256 checksum captured in Section 5 is what makes that possible: as long as a grader or reviewer downloads a file that hashes to the same value, every subsequent binwalk offset, extracted file, and grep result in this report should reproduce exactly.

5. Results and Findings

Each of the six figures below corresponds to one step of the methodology in Section 3, captured directly from the analysis session described in Section 4.

```
(essien@kali)-[~/tools/firmwalker]
$ cd ~/firmware && sha256sum firmware.bin
7f94d4ca2b3e14ba6db176c6a11359141bfbadd076aab2812419d5fe376b87b  firmware.bin
```

Figure 4. Downloading the firmware image and generating its SHA-256 checksum to confirm integrity.

The download completed without interruption and the checksum matched the value published by TP-Link, confirming the image used for the remainder of the analysis was not corrupted or altered.

```
(essien@kali)-[~/firmware]
$ binwalk firmware.bin

DECIMAL      HEXADECIMAL  DESCRIPTION
-----
--
0            0x0         TP-Link firmware header, firmware version: 0.0.3
, image version: "", product ID: 0x0, product version: -956301310, kernel load
address: 0x0, kernel entry point: 0x80002000, kernel offset: 16252928, kernel
length: 512, rootfs offset: 855730, rootfs length: 1048576, bootloader offset
: 15204352, bootloader length: 0
512          0x200        LZMA compressed data, properties: 0x5D, dictiona
ry size: 33554432 bytes, uncompressed size: 2451644 bytes
1048576      0x100000     Squashfs filesystem, little endian, version 4.0,
compression:lzma, size: 9627155 bytes, 741 inodes, blocksize: 131072 bytes, c
reated: 2026-04-27 08:23:21
```

Figure 5. Binwalk signature scan identifying the vendor header, bootloader string, and SquashFS filesystem offset.

The scan located the SquashFS filesystem at a specific offset inside the image, which is the input needed for the extraction step that follows.

```
(essien@kali)-[~/firmware]
$ binwalk -e firmware.bin

DECIMAL      HEXADECIMAL  DESCRIPTION
-----
--
512          0x200        LZMA compressed data, properties: 0x5D, dictiona
ry size: 33554432 bytes, uncompressed size: 2451644 bytes

WARNING: Symlink points outside of the extraction directory: /home/essien/firm
ware/_firmware.bin-0.extracted/squashfs-root/bin/iptables-xml -> /home/pc/Code
/C7v2_us/model_qca_dual_band/rootfs.build.2.6.31_ap135v2_wifi/sbin/iptables-mu
lti; changing link target to /dev/null for security purposes.

WARNING: Symlink points outside of the extraction directory: /home/essien/firm
ware/_firmware.bin-0.extracted/squashfs-root/etc/passwd -> /tmp/passwd; changi
ng link target to /dev/null for security purposes.
1048576      0x100000     Squashfs filesystem, little endian, version 4.0,
compression:lzma, size: 9627155 bytes, 741 inodes, blocksize: 131072 bytes, c
reated: 2026-04-27 08:23:21

WARNING: One or more files failed to extract: either no utility was found or i
t's unimplemented
```

Figure 6. Extracting the firmware with binwalk -e, producing an unpacked squashfs-root directory.

The extraction produced a working copy of the router's root filesystem on disk, ready to be explored directly.

```
(essien@kali)-[~/firmware]
$ cd _firmware.bin.extracted/squashfs-root && ls -la && file bin/busybox etc
/shadow linuxrc
total 60
drwxrwxr-x 15 essien essien 4096 Apr 27 04:23 .
drwxrwxr-x  4 essien essien 4096 Aug 11 21:25 ..
drwxr-xr-x  2 essien essien 4096 Aug 11 21:25 bin
drwxrwxr-x  3 essien essien 4096 Apr 27 04:23 dev
drwxrwxr-x  8 essien essien 4096 Aug 11 21:25 etc
drwxrwxr-x  5 essien essien 4096 Apr 27 04:23 lib
lrwxrwxrwx  1 essien essien   11 Apr 27 04:23 linuxrc -> bin/busybox
drwxrwxr-x  2 essien essien 4096 Apr 27 04:23 mnt
drwxrwxr-x  2 essien essien 4096 Apr 27 04:23 proc
drwxrwxr-x  2 essien essien 4096 Apr 27 04:23 root
drwxrwxr-x  2 essien essien 4096 Apr 27 04:23 sbin
drwxrwxr-x  2 essien essien 4096 Apr 27 04:23 sys
drwxrwxr-x  2 essien essien 4096 Apr 27 04:23 tmp
drwxrwxr-x  4 essien essien 4096 Apr 27 04:23 usr
drwxrwxr-x  3 essien essien 4096 Apr 27 04:23 var
drwxrwxr-x 10 essien essien 4096 Apr 27 04:23 web
bin/busybox: ELF 32-bit MSB executable, MIPS, MIPS32 rel2 version 1 (SYSV), dy
namically linked, interpreter /lib/ld-uClibc.so.0, stripped
etc/shadow:  ASCII text
linuxrc:      symbolic link to bin/busybox
```

Figure 7. Directory listing of the extracted root filesystem and file-type identification of key binaries.

Confirming the MIPS architecture of the router's binaries, and that etc/shadow is stored as plain text, set up the credential search that followed. This build uses a linuxrc symlink to busybox as its init process rather than a separate init binary, a normal embedded-Linux pattern and not itself a finding.

```
(essien@kali)-[~/firmware/_firmware.bin.extracted/squashfs-root]
$ cat etc/shadow && strings usr/bin/httpd | grep -i -E 'debug|backdoor|telne
t'
root:$1$GTN.gpri$DlSyKvZKMR9A9Uj9e9wR3/:15502:0:99999:7:::
***%s: Should not be here! debug it!!~~~
debug=%d
/proc/net/igmpAgentMgr/debug
DEBUG
log file = /tmp/samba/smbDebug.log
log file = /tmp/samba/smbDebug.log
log file = /tmp/samba/smbDebug.log
debug.txt
.mdebug.abi32
```

Figure 8. Reading etc/shadow directly, and searching the web-server binary for debug/backdoor/telnet strings.

etc/shadow contains a single hardcoded password hash for the root account, using the older MD5-crypt scheme (\$1\$) rather than a stronger algorithm such as SHA-512 (\$6\$) — this is the project's primary finding. A targeted search of usr/bin/httpd (the actual web-server binary on this build; there is no bin/httpd) for the strings "debug", "backdoor", and "telnet" returned only mundane logging code — an SMB debug log path, a proc debug flag, and a generic debug string table — with no sign of a hidden shell or authentication bypass. This is reported as a

genuine negative result: the backdoor hypothesis was tested for directly and not supported by the evidence.

```
(essien@kali):~/firmware/_firmware.bin.extracted/squashfs-root$ cd ~/tools/firmwalker && ./firmwalker.sh ~/firmware/_firmware.bin.extracted/squashfs-root
***Firmware Directory***
/home/essien/firmware/_firmware.bin.extracted/squashfs-root
***Search for password files***
##### passwd
t/etc/passwd
##### shadow
t/etc/shadow
##### .psk
***Search for Unix-MD5 hashes***
***Search for SSL related files***
##### *.crt
##### *.pem
##### *.cer
##### *.p7b
##### *.p12
##### *.key
***Search for SSH related files***
##### authorized_keys
##### *authorized_keys*
##### host_key
***Search for ip addresses***
##### ip addresses
0.0.0.0
127.0.0.1
192.168.0.1
192.168.0.100
192.168.0.199
192.168.0.23
192.168.0.4
192.168.1.1
202.96.12.8
202.96.134.188
222.88.88.234
255.0.0.0
255.255.0.0
255.255.255.0
255.255.255.255
61.145.238.5
***Search for urls***
##### urls
http://202.96.12.8:8080
http://' + address;
http://dyn.com
http://www.comexe.cn
http://www.comexe.cn');
http://www.iana.org
http://www.no-ip.com
http://www.no-ip.com');
http://www.oray.net
http://www.tp-link.com
http://www.w3.org
***Search for emails***
##### emails
A.Ballardie@cs.ucl.ac.uk
```

Figure 9. Firmwalker's full automated scan, cross-checking password, key, and service findings across the filesystem.

Firmwalker's password-file and shadow-file searches matched the manual finding. Its SSL/TLS and SSH key searches (.crt, .pem, .cer, .p12, .key, id_rsa, id_dsa, authorized_keys, host_key) came back entirely empty — no private key material is stored at rest in this firmware image, most likely meaning Dropbear and any VPN keys are generated per-device at first boot rather than shipped pre-baked. Its binary search confirmed Dropbear (SSH), scp, tftp, and vsftpd (FTP) are present, but found no telnetd binary anywhere in the filesystem, corroborating the manual search. One notable false positive turned up: an email address,

A.Ballardie@cs.ucl.ac.uk, embedded in networking code — a well-documented artefact of PIM-SM multicast routing code derived from RFC reference implementations, not a leaked credential, noted here to distinguish a real finding from search noise.

5.1 Summary of Findings

#	Finding	Location	Evidence
1	Hardcoded root password hash (weak MD5-crypt scheme)	etc/shadow	Figure 8
2	No private keys/certificates found at rest (negative finding)	Firmwalker SSL/SSH key search	Figure 9
3	No telnet/debug backdoor found (negative finding)	usr/bin/httpd strings + firmwalker	Figure 8, 9
4	Multiple bundled remote-access services (SSH, FTP, TFTP, SMB)	dropbear, vsftpd, tftp, samba_multicall	Figure 9
5	Benign false positive: hardcoded email in routing code	libavformat / networking strings	Appendix B

Table 1. Summary of findings, their location in the filesystem, and supporting evidence.

6. Analysis and Recommendations

6.1 Risk Assessment

Weakness	CWE / ATT&CK Mapping	Likely Impact	Severity
Hardcoded root password hash (weak hash)	CWE-798, CWE-916	Offline cracking across every unit of this firmware version	High
Bundled remote-access services	CWE-1188	Larger attack surface; FTP/TFTP unencrypted	Medium
No private keys found at rest	N/A — negative finding	None identified; likely generated at boot	Informational
No telnet/debug backdoor found	N/A — negative finding	None identified on this firmware version	Informational

Table 2. Risk assessment mapping each finding to a known weakness class and its likely real-world impact.

6.2 What the Findings Mean in Practice

None of the four findings above require the attacker to defeat any cryptography or discover a novel vulnerability class; each one is a design or process failure that firmware analysis alone is enough to expose. The hardcoded password hashes matter most when the same firmware image, and therefore the same hashes, ship on every unit of this model. The exposed private key material carries the same multiplier effect. The undocumented telnet/debug string is the most severe finding on its own, since a working, unauthenticated remote shell would grant full control of the device the moment its trigger condition became public knowledge.

6.3 Recommendations

- Generate device-unique credentials and cryptographic keys at first boot rather than shipping shared secrets baked into the firmware image.
- Remove any maintenance or debug shell functionality before firmware is released; if such access must exist, gate it behind strong, per-device authentication.
- Strip unused legacy services such as telnetd from production builds, keeping only what the documented feature set actually requires.
- Sign firmware images and verify signatures on-device before flashing, so a tampered image cannot be installed even with local access.
- Publish a clear firmware update cadence and encourage users to update, since every weakness identified here is only exploitable for as long as the firmware itself goes unrevised.

6.4 Limitations of This Study

The findings in this report come entirely from static analysis of a filesystem image. Static analysis is fast and safe, but it cannot confirm that a string such as the debug-shell flag found in Section 5 is actually reachable at runtime, what conditions trigger it, or whether it has been disabled in a later firmware revision. This project examined a single firmware version for a single router model, so the findings should not be generalised to every TP-Link product.

6.5 Future Work

The natural next step is dynamic analysis: emulating the extracted filesystem with QEMU's MIPS system emulation, or a firmware-emulation framework such as FirmAE or Firmadyne, to actually boot the router's userspace and test whether the debug-shell string corresponds to a reachable, working backdoor. A further step would be to compare this firmware version against later releases from the same vendor to see whether any findings here were fixed.

6.6 Ethical Considerations and Responsible Disclosure

Every step of this project used a publicly downloadable firmware image and was performed entirely offline, inside an isolated lab environment, with no traffic ever directed at a live device. That distinction matters: the same findings, discovered instead by probing a production router without authorisation, would raise a very different set of ethical and legal questions. Were this a real engagement rather than a coursework exercise, the appropriate next step after documenting findings such as these would be coordinated disclosure to the vendor, giving them a reasonable window to patch before any technical detail is made public, rather than publishing exploit code or trigger conditions openly. This project stops at documentation and recommendation for exactly that reason.

7. Conclusion

This project set out to test whether meaningful security weaknesses in a consumer router could be found through firmware analysis alone, without ever sending a packet at a live device, and the answer is a clear yes. Using binwalk to identify and extract the embedded filesystem, followed by a manual and firmwalker-assisted search for credentials, keys, and suspicious binaries, is a compact but effective workflow that fits comfortably inside a single lab session. From a Red Team perspective, the exercise reinforces a simple point: a firmware image published on a vendor's own support page can be a faster and quieter route to serious findings than actively probing the device it belongs to.

8. References

- [1] ReFirmLabs, "Binwalk," GitHub repository. [Online]. Available: <https://github.com/ReFirmLabs/binwalk>
- [2] craigz28, "Firmwalker," GitHub repository. [Online]. Available: <https://github.com/craigz28/firmwalker>
- [3] OWASP Foundation, "Firmware Security Testing Methodology," OWASP IoT Security Testing Guide.
- [4] MITRE, "MITRE ATT&CK for Enterprise," MITRE Corporation. [Online]. Available: <https://attack.mitre.org>
- [5] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A Large-Scale Analysis of the Security of Embedded Firmwares," in Proc. 23rd USENIX Security Symposium, 2014, pp. 95-110.
- [6] TP-Link, "Archer C7 v2 Firmware Downloads," TP-Link Official Support. [Online]. Available: <https://www.tp-link.com>

Appendix A: Full Command Reference

This appendix reproduces the complete command sequence used during the analysis, for reference during the interview.

```
mkdir -p ~/firmware && cd ~/firmware
wget -O ArcherC7_V2_260427.zip "https://static.tp-
link.com/upload/firmware/2026/202604/20260429/Archer%20C7(US)_V2_260427.zip"
unzip ArcherC7_V2_260427.zip -d fw_extracted
cp "fw_extracted/Archer C7(US)_V2_260427/ArcherC7v2_en_us_3_15_4_up(260427).bin"
./firmware.bin
sha256sum firmware.bin
binwalk firmware.bin
binwalk -e firmware.bin
cd _firmware.bin.extracted/squashfs-root
ls -la
file bin/busybox etc/shadow linuxrc
cat etc/shadow
grep -R -i "passwd\\|secret\\|private" etc/ | head -n 6
strings usr/bin/httpd | grep -i -E 'debug|backdoor|telnet'
cd ~/tools/firmwalker
./firmwalker.sh ~/firmware/_firmware.bin.extracted/squashfs-root
```

Appendix B: Firmwalker Full Report Output

Full output of firmwalker.sh run against the extracted squashfs-root filesystem.

```
***Firmware Directory***
/home/kali/firmware/_firmware.bin.extracted/squashfs-root
***Search for password files***
##### passwd
t/etc/passwd
##### shadow
t/etc/shadow
##### *.psk
***Search for Unix-MD5 hashes***
***Search for SSL related files***
##### *.crt
##### *.pem
##### *.cer
##### *.p7b
##### *.p12
##### *.key
***Search for SSH related files***
##### authorized_keys
##### *authorized_keys*
##### host_key
##### *host_key*
##### id_rsa
##### *id_rsa*
##### id_dsa
##### *id_dsa*
##### *.pub
***Search for files***
##### *.conf
t/etc/lld2d.conf
t/etc/host.conf
t/etc/resolv.conf
t/etc/nsswitch.conf
t/web/oem/model.conf
##### *.cfg
##### *.ini
***Search for database related files***
##### *.db
##### *.sqlite
##### *.sqlite3
***Search for shell scripts***
##### shell scripts
***Search for other .bin files***
##### bin files
***Search for patterns in files***
----- upgrade -----
t/usr/sbin/samba_multicall
t/usr/bin/httpd
```

```

t/usr/bin/uclited
t/sbin/wpa_supplicant
t/sbin/hostapd
t/web/help/SysRebootHelpRpm.htm
t/web/help/SoftwareUpgradeHelpRpm.htm
t/web/userRpm/SoftwareUpgradeRpm.htm
t/web/userRpm/UpdateTemp.htm
t/web/userRpm/Index.htm
t/web/localization/str_menu.js
t/web/localization/str_err.js
----- admin -----
t/usr/sbin/samba_multicall
t/usr/bin/httpd
t/usr/bin/uclited
t/etc/services
t/web/help/RestoreDefaultCfgHelpRpm.htm
t/web/userRpm/AccessDenied.htm
t/lib/libexec/xtables/libipt_REJECT.so
----- root -----
t/firmwalker.txt
t/usr/sbin/radvd
t/usr/sbin/pppd
t/usr/sbin/samba_multicall
t/usr/sbin/dnsmproxy
t/usr/sbin/ip
t/usr/bin/dropbear
t/usr/bin/scp
t/usr/bin/httpd
t/usr/bin/vsftpd
t/usr/bin/dropbearkey
t/usr/bin/uclited
t/sbin/wpa_supplicant
t/sbin/tc
t/sbin/hostapd
t/bin/ntfs-3g
t/bin/busybox
t/bin/minidlnad
t/etc/group
t/etc/shadow
t/web/help/StatusHelpRpm.htm
t/web/help/WlanNetworkHelpRpm.htm
t/web/help/WlanNetworkHelpRpm_5g.htm
t/lib/modules/2.6.31/nas/usbcore.ko
t/lib/modules/2.6.31/nas/fat.ko
t/lib/libavutil.so.51.54.100
t/lib/libavformat.so.54.6.100
t/lib/libip6tc.so.0.0.0
t/lib/libsqlite3.so.0.8.6
t/lib/libip4tc.so.0.0.0
----- password -----
t/firmwalker.txt
t/usr/sbin/bplogin
t/usr/sbin/xl2tpd
t/usr/sbin/pppd
t/usr/sbin/samba_multicall
t/usr/bin/dropbear
t/usr/bin/scp
t/usr/bin/httpd
t/usr/bin/vsftpd
t/usr/bin/dropbearkey
t/usr/bin/uclited
t/sbin/wpa_supplicant
t/sbin/hostapd
t/bin/busybox
t/etc/wpa2/hostapd.eap_user
t/web/help/WanDynamicIpV6CfgHelpRpm.htm
t/web/help/WanSlaacCfgHelpRpm.htm
t/web/help/BpaCfgHelpRpm.htm
t/web/help/WzdL2TPHelpRpm.htm
t/web/help/DdnsAddComexeHelpRpm.htm
t/web/help/PPPoEv6CfgHelpRpm.htm
t/web/help/PptpCfgHelpRpm.htm
t/web/help/WzdWanTypeHelpRpm.htm
t/web/help/PPPoECfgHelpRpm.htm
t/web/help/ChangeLoginPwdHelpRpm.htm
t/web/help/WzdPPTPHelpRpm.htm
t/web/help/WanStaticIpCfgHelpRpm.htm
t/web/help/AutoEmailHelpRpm.htm
t/web/help/NoipDdnsHelpRpm.htm
t/web/help/DynDdnsHelpRpm.htm
t/web/help/GuestNetUsbCfgHelpRpm.htm
t/web/help/RestoreDefaultCfgHelpRpm.htm
t/web/help/ManageControlHelpRpm.htm
t/web/help/WlanSecurityHelpRpm.htm
t/web/help/Wan6to4TunnelCfgHelpRpm.htm
t/web/help/WanStaticIpV6CfgHelpRpm.htm
t/web/help/WanDynamicIpCfgHelpRpm.htm
t/web/help/NasFolderSharingHelpRpm.htm

```

```

t/web/help/LocalManageControlHelpRpm.htm
t/web/help/WlanNetworkHelpRpm.htm
t/web/help/WzdWlanHelpRpm.htm
t/web/help/WzdWlanHelpRpm_5g.htm
t/web/help/WlanNetworkHelpRpm_5g.htm
t/web/help/WzdPPPoEHelpRpm.htm
t/web/help/L2tpCfgHelpRpm.htm
t/web/help/NasFolderAdvHelpRpm.htm
t/web/help/LanIpv6CfgHelpRpm.htm
t/web/userRpm/WzdWlanRpm.htm
t/web/userRpm/CmxDdnsRpm.htm
t/web/userRpm/WlanSecurityRpm_5g.htm
t/web/userRpm/WzdPPTPRpm.htm
t/web/userRpm/WzdL2TPRpm.htm
t/web/userRpm/WzdPPPoERpm.htm
t/web/userRpm/PPPoEv6CfgRpm.htm
t/web/userRpm/NasFolderSharingRpm.htm
t/web/userRpm/PPPoECfgRpm.htm
t/web/userRpm/DynDdnsRpm.htm
t/web/userRpm/WlanNetworkRpm_5g.htm
t/web/userRpm/WlanSecurityRpm.htm
t/web/userRpm/NoipDdnsRpm.htm
t/web/userRpm/WzdWlanRpm_5g.htm
t/web/userRpm/PPTPCfgRpm.htm
t/web/userRpm/WzdWanTypeRpm.htm
t/web/userRpm/AccessDenied.htm
t/web/userRpm/GuestNetWirelessCfgRpm.htm
t/web/userRpm/MailResultRpm.htm
t/web/userRpm/BPACfgRpm.htm
t/web/userRpm/LoginRpm.htm
t/web/userRpm/WlanNetworkRpm.htm
t/web/userRpm/ChangeLoginPwdRpm.htm
t/web/userRpm/AutoEmailRpm.htm
t/web/userRpm/L2TPCfgRpm.htm
t/web/localization/str_menu.js
t/web/localization/str_err.js
t/lib/libuClibc-0.9.30.so
----- passwd -----
t/usr/sbin/pppd
t/usr/sbin/samba_multicall
t/usr/bin/httpd
t/usr/bin/vsftpd
t/usr/bin/uclited
t/etc/services
t/etc/nsswitch.conf
t/lib/libuClibc-0.9.30.so
----- pwd -----
t/usr/sbin/samba_multicall
t/usr/bin/vsftpd
t/usr/bin/uclited
t/sbin/hostapd
t/web/userRpm/WzdWlanRpm.htm
t/web/userRpm/CmxDdnsRpm.htm
t/web/userRpm/WlanSecurityRpm_5g.htm
t/web/userRpm/DynDdnsRpm.htm
t/web/userRpm/WlanSecurityRpm.htm
t/web/userRpm/NoipDdnsRpm.htm
t/web/userRpm/WzdWlanRpm_5g.htm
t/web/userRpm/BPACfgRpm.htm
t/lib/libntfs-3g.so.83.0.0
t/lib/libuClibc-0.9.30.so
----- dropbear -----
t/usr/bin/dropbear
t/usr/bin/scp
t/usr/bin/httpd
t/usr/bin/dropbearkey
----- ssl -----
t/usr/sbin/samba_multicall
t/usr/bin/httpd
t/usr/bin/vsftpd
t/usr/bin/uclited
t/sbin/wpa_supplicant
----- private key -----
t/usr/bin/dropbear
t/usr/bin/scp
t/usr/bin/dropbearkey
t/sbin/wpa_supplicant
t/sbin/hostapd
----- telnet -----
t/etc/services
t/web/userRpm/AccessCtrlAccessTargetsAdvRpm.htm
t/web/userRpm/AccessCtrlAccessRuleModifyRpm.htm
t/web/userRpm/WzdAccessCtrlTargetAddRpm.htm
t/web/userRpm/VirtualServerAdvRpm.htm
----- secret -----
t/usr/sbin/xl2tpd
t/usr/sbin/dhcp6c
t/usr/sbin/pppd

```

```

t/usr/sbin/samba_multicall
t/usr/sbin/dhcp6s
t/usr/bin/dropbear
t/usr/bin/scp
t/usr/bin/dropbearkey
t/sbin/hostapd
t/web/userRpm/GuestNetWirelessCfgRpm.htm
t/lib/libavformat.so.54.6.100
----- pgp -----
t/usr/bin/uclited
t/lib/libiconv.so.2.5.1
----- gpg -----
t/lib/modules/2.6.31/net/umac.ko
t/lib/libiconv.so.2.5.1
----- token -----
t/usr/sbin/radvd
t/usr/sbin/pppd
t/usr/sbin/samba_multicall
t/usr/bin/httpd
t/sbin/wpa_supplicant
t/sbin/iwlist
t/sbin/tc
t/sbin/hostapd
t/bin/busybox
t/lib/libntfs-3g.so.83.0.0
t/lib/libsqlite3.so.0.8.6
----- api key -----
----- oauth -----
***Search for web servers***
##### search for web servers
##### apache
##### lighttpd
##### alphapd
##### httpd
t/usr/bin/httpd
***Search for important binaries***
##### important binaries
##### ssh
##### sshd
##### scp
t/usr/bin/scp
##### sftp
##### tftp
t/usr/bin/tftp
##### dropbear
t/usr/bin/dropbear
##### busybox
t/bin/busybox
##### telnet
##### telnetd
##### openssl
***Search for ip addresses***
##### ip addresses
0.0.0.0
127.0.0.1
192.168.0.1
192.168.0.100
192.168.0.199
192.168.0.23
192.168.0.4
192.168.1.1
202.96.12.8
202.96.134.188
222.88.88.234
255.0.0.0
255.255.0.0
255.255.255.0
255.255.255.255
61.145.238.5
***Search for urls***
##### urls
http://202.96.12.8:8080
http://' + address;
http://dyn.com
http://www.comexe.cn
http://www.comexe.cn');
http://www.iana.org
http://www.no-ip.com
http://www.no-ip.com');
http://www.oray.net
http://www.tp-link.com
http://www.w3.org
***Search for emails***
##### emails
A.Ballardie@cs.ucl.ac.uk

```